

# Digital Circuits Modelling using **VERILOG**

**Dr. Chandan Karfa**

**Department of Computer Science and Engineering**



भारतीय प्रौद्योगिकी संस्थान गुवाहाटी  
INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI



# Reference Book

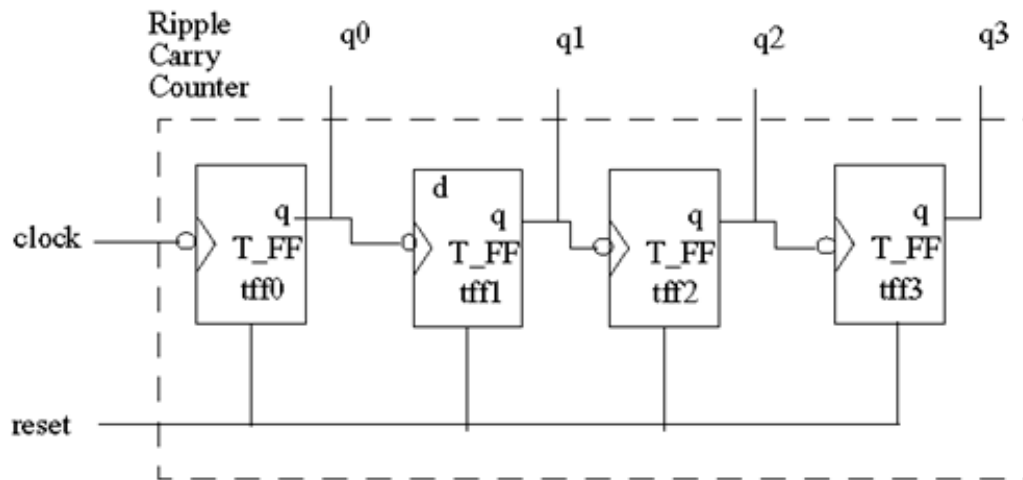
- S. Palnitkar, Verilog HDL: A Guide to Digital Design and Synthesis, Pearson, 2nd Ed, 2003.

## Acknowledgement:

- Most of the examples of Verilog Lectures are taken from this book

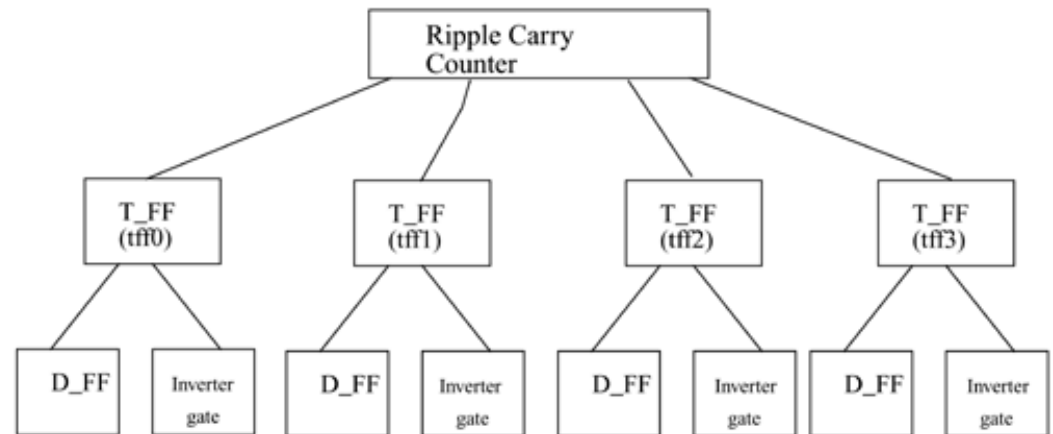
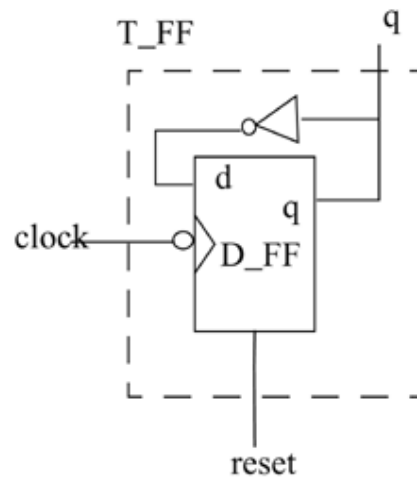
# Hierarchical Design Concept

- The entire design breaks into sub-modules.
  - Each sub-modules can also be break into sub-modules and so on...



# Hierarchical Design Concept

| reset | $q_n$ | $q_{n+1}$ |
|-------|-------|-----------|
| 1     | 1     | 0         |
| 1     | 0     | 0         |
| 0     | 0     | 1         |
| 0     | 1     | 0         |
| 0     | 0     | 0         |



```
module mux41 ( in0, in1, in2, in3, sel0, sel1, out );
```

→ Module Name  
& Ports List

```
input sel0, sel1;
```

```
input in0, in1, in2, in3;
```

```
output out;
```

```
reg op;
```

→ Declarations

```
always @( in0 or in1 or in2 or in3 or sel0 or sel1 )  
begin
```

```
    case ( { sel1, sel0 } )
```

```
        2'b00: op <= in0;
```

```
        2'b01: op <= in1;
```

```
        2'b10: op <= in2;
```

```
        2'b11: op <= in3;
```

```
        default: op <= op;
```

```
    endcase
```

```
end
```

```
assign out = op;
```

→ Statements

```
endmodule
```

→ End of  
module

# Structure of a Module

```
module <mod name> (<port list>);
    <declarations>; // input, output, inout
                    // wire, register, etc.
    <statements>;  // initial, begin, end, always
                    // dataflow statements
endmodule
```

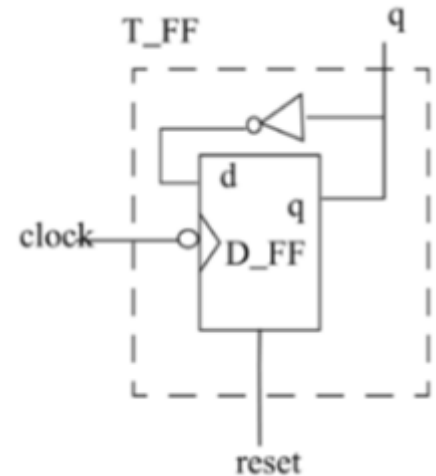
# Ports

- Ports provide interface by which a module can communicate with its environment.
  - By default ports are wire/net
    - input** – Input port
    - output** – Output port
    - inout** – Bidirectional port
- Default type for input/output/inout is “**wire**”.
  - “**input wire in\_1**” is same as “**input in\_1**”.
  - “**output wire out\_1**” is same as “**output out\_1**”.

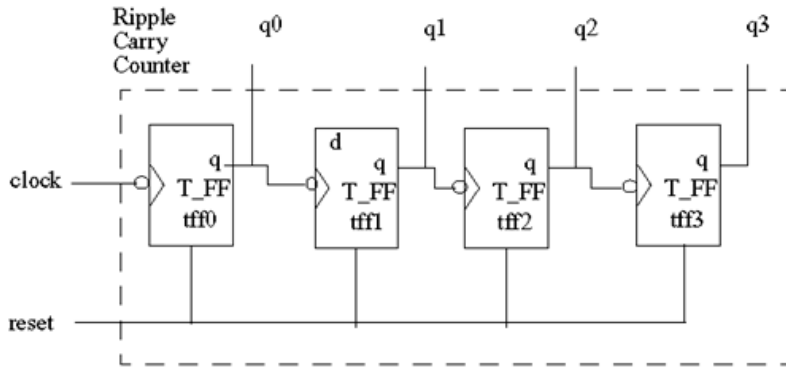
# Module Instantiations

- Module defines a template
- Verilog creates a unique object from the template through module Instantiation

```
module T_FF(q, clk, reset);  
    output q;  
    input clk, reset;  
    wire d;  
    D_FF dff0(q, d, clk, reset); //  
    Instantiate D_FF. Call it dff0.  
    not n1(d, q); // not gate is a Verilog  
    primitive. Explained later.  
endmodule
```



# Module Instantiations



```
module ripple_carry_counter(q, clk, reset);  
    output [3:0] q; //I/O signals and vector declarations  
    input clk, reset; //I/O signals will be explained later.  
    T_FF tff0(q[0], clk, reset);  
    T_FF tff1(q[1], q[0], reset);  
    T_FF tff2(q[2], q[1], reset);  
    T_FF tff3(q[3], q[2], reset);  
endmodule
```

# Value Set in Verilog

| Value Level | Condition in Hardware Circuits |
|-------------|--------------------------------|
| 0           | Logic zero, false condition    |
| 1           | Logic one, true condition      |
| x           | Unknown logic value            |
| z           | High impedance, floating state |

- X and Z value are undesired values

# Basic Syntax

- Comments

`a = b && c; // This is a one-line comment`  
`/* This is a multiple line  
comment */`

- Operators

Unary Binary Ternary

`a = ~ b; // ~ is a unary operator. b is the operand`

`a = b && c; // && is a binary operator.`

`a = b ? c : d; // ?: is a ternary operator`

# Number Specification

`<size> '<base format> <number>`

## Examples:

- `4'b1111` // This is a 4-bit binary number
- `12'habc` // This is a 12-bit hexadecimal number
- `16'd255` // This is a 16-bit decimal number.
- (d or D), (h or H), (b or B) are allowed

# Number Specification

- Numbers that are specified without a <base format> specification are decimal numbers by default
- Numbers that are written without a <size> specification have a default number of bits that is simulator- and machine-specific (must be at least 32).
- 23456 // This is a 32-bit decimal number by default
- 'hc3 // This is a 32-bit hexadecimal number
- 'o21 // This is a 32-bit octal number

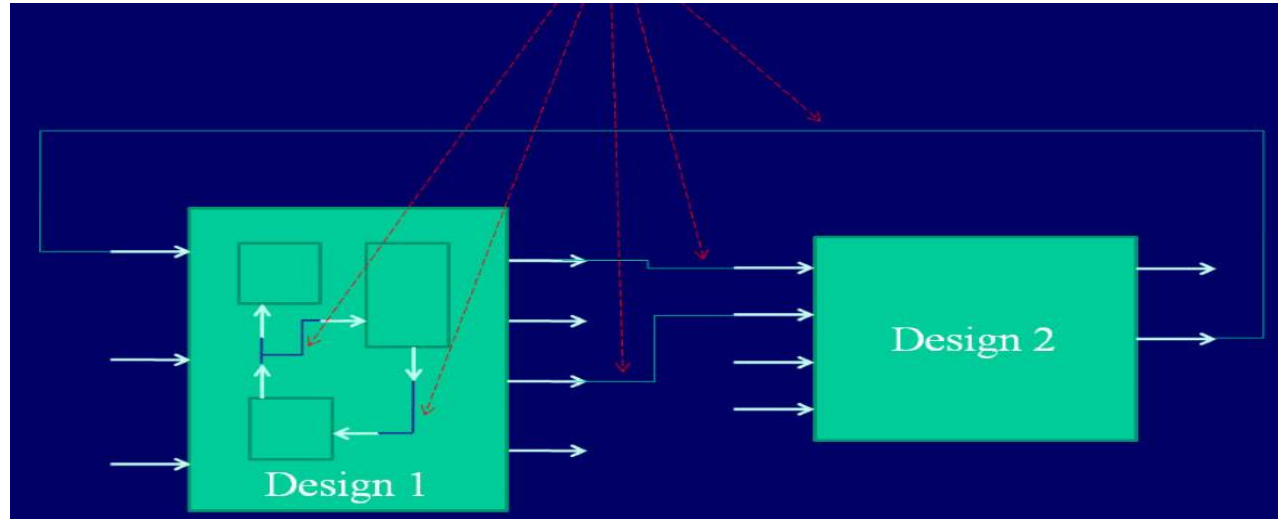
# Negative Number

- Negative numbers can be specified by putting a minus sign before the size for a constant number.
- `-6'd3` // 6-bit negative number stored as 2's complement of 3
- `4'd-2` // Illegal specification

# Verilog Components

- Net
  - Make connections between components
- Registers
  - Represents Data storage Elements
- Memories
  - Array of data storage

# Nets



- Nets represent connections between hardware elements.
- They are always driven by some source.
- Declared by keyword **“wire”**.
- It can hold any one of the four values: 0, 1, x, z. Default value for any net type variable is ‘z’ (High impedance – hanging net).
- Always need a **driver**.

# Registers

- Register data types always retain their value until another value is placed on them.
- Unlike nets, registers do not need any drivers.
- Declared by keyword “**reg**”.
- It can hold any one of the four values: 0, 1, x, z. Default value for any reg type variable is ‘x’ (unknown).
- reg is not same as flip flop

# Vectors

- Nets or reg data types can be declared as vectors (multiple bit widths).
- If bit width is not specified, the default is scalar (1-bit).

```
wire a; // scalar net variable, default
wire [7:0] bus; // 8-bit bus
wire [31:0] busA,busB,busC; // 3 buses of 32-bit width.
reg clock; // scalar register, default
reg [0:40] virtual_addr; /* Vector register, virtual
address 41 bits wide */
```

# Vectors

- Vectors can be declared at [high# : low#] or [low# : high#],
- But the left number in the squared brackets is always the most significant bit of the vector.
- In the example shown above, bit 0 is the most significant bit of vector `virtual_addr`.

# Vectors – Part Select

- `wire [7:0] bus; // 8-bit bus`
- `reg [0:40] virtual_addr;`
- For the vector declarations shown above, it is possible to address bits or parts of vectors.
  - `busA[7]` // bit # 7 of vector `busA`
  - `bus[2:0]` // Three least significant bits of vector `bus`,
  - using `bus[0:2]` is illegal because the significant bit should always be on the left of a range specification
  - `virtual_addr[0:1]` // Two most significant bits of vector `virtual_addr`



# Memories

- Often needs to model register files, RAMs, and ROMs.
- Memories are modeled in Verilog simply as a one-dimensional array of registers

**reg [wordsize:0] memory [0:arraysize]**

- reg mem1bit[0:1023]; // Memory mem1bit with 1K 1-bit words
- reg [7:0] membyte[0:1023]; // Memory membyte with 1K 8-bit words(bytes)



# Parameters

- A parameter is defined by Verilog as a constant value declared within the module.

- Syntax:

parameter <identifier> = constant;

- Example:

```
parameter byte_size = 8;  
reg[byte_size-1:0] A;
```

```
wire a ; //1-bit wire
wire [7:0] bus; //8-bit bus
wire [31:0] busA, busB, busC ; //3 buses of 32-bit width
```

```
reg a; //1-bit register
reg [31:0] b; //32-bit register
```

- **Memories** (array of registers)  
    **reg [wordsize:0] memory [0:arraysize]**  
        reg [3 : 0] mem\_a [63:0] ; //sixty-four 4-bit registers  
        mem\_a [address] = data\_in ;  
        reg mem\_b [0 : 4] // five 1-bit registers
- **Number format**  
    **<size> ' <base\_format> <number\_format>**  
    4'b1111   //This is a 4-bit binary number  
    12'habc   //This is a 12-bit hexadecimal number  
    16'd255   //This is a 16-bit decimal number  
    23456     //This is a 32-bit decimal number by default  
    `hc3       //This is a 32-bit hexadecimal number  
    `o21       //This is a 32-bit octal number

Thank You